2026 IEEE 8th International Conference on BioInspired Processing (BIP) - #71710, Guápiles, Costa Rica. November 11th–13th, 2026.

Formal Specification and Extended Evaluation of a Bio-Inspired Hyperparameter Optimization Framework for Software Defect Density Prediction

Juan Murillo-Morera[1], Rubén Fuentes-Fernández[2], Andrés Calvo Vargas[3], Alessandro Cambronero Delgado[3], Janio Jadán-Guerrero[4], Susana Alexandra Arias[5]

[1]School of Informatics and Computing, National University of Costa Rica
[2]Institute of Knowledge Technology & Research Group on Agent-based, Social Interdisciplinary Applications,
University Complutense of Madrid, Spain
[3]School of Informatics and Computing, National University of Costa Rica
[4]Research Center for Human Sciences and Education (CICHE), Indo-America Technological University, 170103, Quito, Ecuador
[5]International University of La Rioja, Spain

[1]juan.murillo.morera@una.cr, [2]ruben@fdi.ucm.es, [3]{andres.calvo.vargas, alessandro.cambronero.delgado}@est.una.ac.cr,
[4]janiojadan@uti.edu.ec, [5]susanaalexandra.arias-externo@unir.net

***Abstract*—Software defect density prediction is challenged by heterogeneous data and the computational cost of hyperparameter optimization. Justification: BIP 2025 introduced a Generator–Evaluator–Predictor framework, but its formal specification and evaluation with multiple bio-inspired optimizers remained open. Objective: Formally extend the framework and compare GA, PSO, ABC, CDEA, GWO, and WOA. Method: Ten PROMISE datasets, eleven regression models, and 660 dataset–model–optimizer combinations were evaluated using inferential statistics. Results: Execution time differs significantly ($p < 0.001$, $W = 0.377$), while global RMSE differences are not significant ($p = 0.058$, $W = 0.019$). Configuration dispersion is strongly associated with RMSE dispersion ($\rho = 0.622$, $p < 0.001$). Conclusion: The framework supports modular optimizer selection considering computational cost, configuration robustness, and predictive performance. Future work: Analyze optimizer trajectories and more datasets more extensively.**

***Index Terms*—Software defect density prediction, empirical SE, hyperparameter optimization, bio-inspired algorithms, formal specification, sensitivity analysis**

I. Introduction

Software defect density prediction supports software quality assurance by estimating defects from software metrics, but heterogeneous and noisy datasets make model construction difficult [1], [2]. Hyperparameter optimization can improve regression models, although alternative search strategies may incur very different computational costs even when predictive performance is similar.

This paper is a direct second-stage continuation of the framework introduced and empirically validated at IEEE BIP 2025 [3]. It deliberately preserves the published Generator–Evaluator–Predictor Black-Box architecture, its Data Flow–Optimization–Prediction White-Box realization, and the original experimental contract. BIP 2026 extends that baseline in three directions: (i) a hierarchical formal specification of its components, relations, operators, constraints, and state transitions; (ii) generalization of the GA-centered Optimization component to GA, PSO, ABC, CDEA, GWO, and WOA; and (iii) broader empirical validation over ten PROMISE datasets, eleven regressors, and 660 dataset–model–optimizer records. An original overview of the extended framework is provided in Fig. 1, complemented by a simplified Generator–Evaluator–Predictor interaction view in Fig. 2.

Following GQM [4], we compare predictive accuracy, configuration–output sensitivity, and computational cost under a fixed Generator–Evaluator–Predictor contract. This isolates optimizer substitution from preprocessing and evaluation changes, while distinguishing architectural formalization from empirical validation. The remainder presents the RQs, related work, formal model, protocol, results, discussion, threats, and conclusions.

979-8-3195-4761-3/26/$31.00 © 2026 IEEE

II. Research Questions

This study is guided by four research questions:

- **RQ1:** What is the execution time and computational performance of bio-inspired algorithms when applied to different datasets?
- **RQ2:** How sensitive is predictive performance to differences in the hyperparameter configurations selected by the bio-inspired optimization algorithms?
- **RQ3:** Which bio-inspired algorithm achieves the best observed predictive performance for each dataset, and are the observed differences statistically significant?
- **RQ4:** Are there statistically significant differences in RMSE among the evaluated bio-inspired optimization algorithms?

III. Related Work

Software defect prediction has been studied with statistical and machine-learning models, with prior work emphasizing rigorous comparison, data quality, feature selection, preprocessing, and defect-proneness frameworks [5], [6], [7], [8], [9], [10], [1], [11], [2], [12], [13], [14]. Hyperparameter optimization adds another source of variability because the search space combines preprocessing, learners, and model-specific parameters [15]. Bio-inspired search provides practical alternatives to exhaustive tuning and has been applied in earlier prediction frameworks [16], [17]; relevant families include GA [18], [19], PSO [20], ABC [21], Differential Evolution variants [22], GWO [23], and WOA [24].

Research gap and continuity. BIP 2025 [3] established the Generator–Evaluator–Predictor framework and empirically validated a GA-centered realization. The present study does not replace that architecture. It asks how the inherited Optimization component behaves when instantiated with six bio-inspired mechanisms under the same data-flow and evaluation contracts, while additionally formalizing the framework and expanding the empirical evidence. Thus, the novelty is the combination of formal specification, algorithmic generalization, and extended comparative validation.

IV. Theoretical Framework

This section formalizes the framework introduced at IEEE BIP 2025 [3]; it does not redefine its architecture. The predecessor supplies the Black-Box components *Generator–Evaluator–Predictor* and the White-Box processes *Data Flow–Optimization–Prediction*. BIP 2026 preserves this architecture, generalizes the Optimization component to six bio-inspired algorithms, and supplies a compact hierarchical mathematical specification. The general structure of the framework is illustrated in Fig. 1.

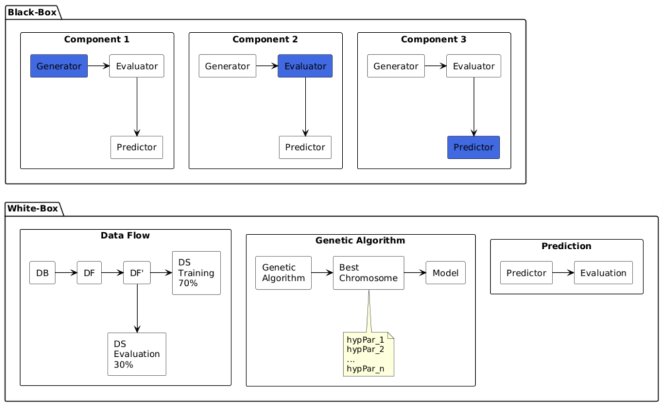


Fig. 1. Overview of the bio-inspired hyperparameter optimization framework for software defect density prediction.

To make the notation operationally readable, the five definitions follow the execution path rather than introducing independent abstractions: Definition 1 states the complete contract; Definition 2 identifies what can be generated and optimized; Definition 3 states how candidates are evaluated; Definition 4 isolates final prediction on unseen data; and Definition 5 links these stages. In short, $\mathcal{G}$ proposes a learner configuration, $\mathcal{E}$ scores it using training-only cross-validation, and $\mathcal{P}$ fits the selected configuration and evaluates it once on the held-out test partition. Symbols used only to express this contract are introduced locally below.

Definition 1 (Framework). Let $\mathfrak{F}$ be defined as

$$\mathfrak{F} = \langle \mathcal{G}, \mathcal{E}, \mathcal{P}, \mathcal{R}, \Phi, \Omega \rangle, \qquad \mathcal{C}_{BB} = \{\mathcal{G}, \mathcal{E}, \mathcal{P}\},$$

where $\mathcal{G}$, $\mathcal{E}$, and $\mathcal{P}$ are the Generator, Evaluator, and Predictor sets; $\mathcal{R}$ contains the relations among their elements; Φ contains data/model transformations; and Ω contains the invariants of the experimental contract.

Definition 2 (Generator). The Generator is the nested set

$$\mathcal{G} = \{\mathcal{D}, \mathcal{A}, \mathcal{L}, \mathcal{H}, \mathcal{O}, \mathcal{B}, \mathcal{U}\},$$

with

$$\begin{aligned}
\mathcal{D} &= \{d_1, \ldots, d_{10}\}, \quad d = (X, y), \\
\mathcal{A} &= \{\texttt{SelectFromModel(LassoCV)}\}, \\
\mathcal{L} &= \{LR, DT, RF, GB, LASSO, RIDGE, \\
&\qquad EN, KNN, SVR, MLP, XGB\}, \\
\mathcal{H} &= \bigcup_{\ell \in \mathcal{L}} \mathcal{H}_\ell, \quad \mathcal{H}_\ell = \prod_{j=1}^{q_\ell} H_{\ell j}, \\
\mathcal{O} &= \{GA, PSO, ABC, CDEA, GWO, WOA\}, \\
\mathcal{B}_o^{(t)} &= \{h_{o,1}^{(t)}, \ldots, h_{o,N_o}^{(t)}\}, \\
\mathcal{U} &= \{\mathcal{U}_{GA}, \mathcal{U}_{PSO}, \mathcal{U}_{ABC}, \\
&\qquad \mathcal{U}_{CDEA}, \mathcal{U}_{GWO}, \mathcal{U}_{WOA}\}.
\end{aligned}$$

The operator subsets are

$$\begin{aligned}
\mathcal{U}_{GA} &= \{selection, crossover, mutation, elitism\},\\
\mathcal{U}_{PSO} &= \{position, pbest, gbest, update\},\\
\mathcal{U}_{ABC} &= \{employed, onlooker, scout, reinit\},\\
\mathcal{U}_{CDEA} &= \{variation, crossover, greedy\},\\
\mathcal{U}_{GWO} &= \{leaders, encircling, hunting\},\\
\mathcal{U}_{WOA} &= \{encircling, spiral, exploration\}.
\end{aligned}$$

Thus the hierarchy is explicit, e.g. $\mathcal{G} \supset \mathcal{U} \supset \mathcal{U}_{GA} \ni elitism$.

Definition 3 (Evaluator). The Evaluator is

$$\mathcal{E} = \{\mathcal{K}, \mathcal{J}, \mathcal{M}, \mathcal{Q}, \mathcal{V}\},$$

where $\mathcal{K} = \{K_1, \ldots, K_{10}\}$ is the 10-fold partition, $\mathcal{J}$ is the set of candidate fitness values, $\mathcal{M} = \{MSE, RMSE, MAE, R^2, T_{exec}\}$, $\mathcal{Q}$ is the ranking/preference structure, and $\mathcal{V}$ is the survivor/best-memory set. The basic evaluation relation is

$$R_{eval} \subseteq \mathcal{H} \times \mathcal{K} \times \mathcal{M} \times \mathbb{R},$$

and predictive preference is $h_i \prec_J h_j \iff J_{cv}^{MSE}(h_i) < J_{cv}^{MSE}(h_j)$.

Definition 4 (Predictor). The Predictor is

$$\mathcal{P} = \{\mathcal{H}^*, \mathcal{T}, \mathcal{F}, \mathcal{X}_{te}, \widehat{\mathcal{Y}}, \mathcal{M}_{te}\},$$

where $\mathcal{H}^* = \{h^*\}$ is the selected configuration, $\mathcal{T}$ the transformed training data, $\mathcal{F} = \{\widehat{f}_{\ell,h^*}\}$ the fitted model, $\mathcal{X}_{te}$ the held-out test data, $\widehat{\mathcal{Y}}$ the predictions, and $\mathcal{M}_{te} = \{RMSE_{te}, MAE_{te}, R^2_{te}\}$.

Definition 5 (Relations). The structural and operational relations are

$$\mathcal{R} = \{R_{contains}, R_{GE}, R_{EG}, R_{EP}, R_{fit}, R_{pred}\},$$

with $R_{contains}$ encoding the nested membership hierarchy, $R_{GE} \subseteq \mathcal{G} \times \mathcal{E}$ candidate-to-fitness flow, $R_{EG} \subseteq \mathcal{E} \times \mathcal{G}$ fitness feedback, $R_{EP} \subseteq \mathcal{E} \times \mathcal{P}$ best-configuration selection, $R_{fit} : (h^*, \mathcal{T}) \mapsto \widehat{f}$, and $R_{pred} : (\widehat{f}, \mathcal{X}_{te}) \mapsto \widehat{\mathcal{Y}}$. These relations are the formal counterpart of the White-Box sequence inherited from BIP 2025.

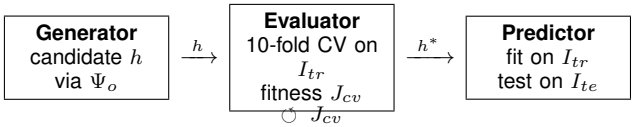


Fig. 2. Simplified Generator–Evaluator–Predictor interaction. The feedback loop updates the optimizer only from training-fold fitness; the held-out test partition is accessed only after selecting h^*.

A. Data, Selection, and Cross-Validation

For pass $r \in \{1, \ldots, 10\}$, the data indices satisfy

$$I_{tr}^{(r)} \cap I_{te}^{(r)} = \varnothing, \qquad |I_{tr}^{(r)}| \simeq 0.70n, \quad |I_{te}^{(r)}| \simeq 0.30n.$$

The response transformation is $g(y) = \log(y+1)$. With training mean μ and $S = \mathrm{diag}(\sigma_1, \ldots, \sigma_p)$, standardization is

$$\mathcal{Z}(X) = (X - \mathbf{1}\mu^\top)S^{-1}. \tag{1}$$

Feature selection is defined by

$$\begin{aligned}
\widehat{\beta}(\alpha) &= \arg\min_{\beta} \left\{ \frac{1}{2n}\|y - X\beta\|_2^2 + \alpha\|\beta\|_1 \right\},\\
M &= \{j : \widehat{\beta}_j(\alpha^*) \neq 0\},
\end{aligned}$$

where α^* is selected by LassoCV and $\Pi_M(X) = X[:, M]$. The internal folds are constructed *only* from the outer training indices: $K_k \subset I_{tr}^{(r)}$ for every k, $\bigcup_k K_k = I_{tr}^{(r)}$, and $K_k \cap I_{te}^{(r)} = \varnothing$. Preprocessing parameters and feature selection are fitted within training data and are never estimated from $I_{te}^{(r)}$. For candidate h, the optimization objective follows the implementation exactly:

$$J_{cv}^{MSE}(h) = \frac{1}{10}\sum_{k=1}^{10} MSE\Big(y_{K_k}, \widehat{f}_h^{(-k)}(X_{K_k})\Big). \tag{2}$$

The implementation maximizes the negative of Eq. (2), hence it is exactly equivalent to minimizing mean cross-validated MSE. RMSE is *not* substituted inside this fold-averaged optimization objective: although $RMSE = \sqrt{MSE}$ for a fixed set of residuals and the square root is monotone, averaging fold-wise RMSE is not generally rank-equivalent to averaging fold-wise MSE. Final predictive performance is therefore reported separately as $RMSE_{te}$ on the held-out test partition.

B. Learning Models and Hyperparameter Spaces

Rather than assigning a separate formal definition to every learner, all learners are instances of the common map

$$\mathcal{T}_\ell : (X^{tr}, y^{tr}, h) \mapsto \widehat{f}_{\ell,h}, \qquad h \in \mathcal{H}_\ell.$$

Their mathematical cores are summarized below; the experimental hyperparameter domains are those reported in Sec. V.

TABLE I
COMPACT MATHEMATICAL REPRESENTATION OF THE LEARNING ALGORITHMS.

Learner	Mathematical core
LR	$\widehat{y} = X\beta + b$; least-squares estimation.
Lasso/Ridge/EN	$\min_\beta \frac{1}{2n}\|y - X\beta\|_2^2 + \lambda R_\eta(\beta)$, where $R_\eta = \eta\|\beta\|_1 + (1-\eta)\|\beta\|_2^2$.
DT	Recursive split (j, s) minimizing within-node squared error.
RF	$\widehat{f}(x) = B^{-1}\sum_{b=1}^{B} T_b(x)$.
GB	$F_m(x) = F_{m-1}(x) + \nu\gamma_m T_m(x)$.
KNN	$\widehat{f}(x) = \sum_{i\in N_k(x)} w_i y_i / \sum_{i\in N_k(x)} w_i$.
SVR	$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_i(\xi_i + \xi_i^*)$ under ϵ-tube constraints.
MLP	$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$.
XGB	$\widehat{y}_i = \sum_{m=1}^{M} f_m(x_i)$ with regularized additive-tree objective.

C. Unified Bio-Inspired Optimization Model

All optimizers act on the same learner-specific search domain $\mathcal{H}_\ell$. Let optimizer state s_t^o contain its popula-

tion/swarm/colony, memory, and control parameters. A single transition schema covers all six methods:

$$s^o_{t+1} = \Psi_o(s^o_t, J^{MSE}_{cv}), \qquad \mathcal{B}^{(t+1)}_o \subseteq \mathcal{H}_\ell,$$

where Ψ_o differs only in its search operator. The canonical update defining each Ψ_o is:

TABLE II
OPTIMIZER-SPECIFIC REALIZATION OF THE COMMON TRANSITION Ψ_o.

OA	Canonical search relation
GA	$B_{t+1} = Elite(B_t) \cup Mutation(Crossover(Select(B_t)))$.
PSO	$v' = \omega v + c_1 r_1 (p - x) + c_2 r_2 (g - x)$; position update $x' = x + v'$.
ABC	$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj})$; employed/onlooker selection plus scout reinitialization.
CDEA	$v = x_{r_1} + F(x_{r_2} - x_{r_3})$ followed by crossover and greedy replacement.
GWO	$x_{t+1} = (x_1 + x_2 + x_3)/3$, where x_1, x_2, x_3 are induced by α, β, δ leaders.
WOA	$x_{t+1} = x^* - A\lvert Cx^* - x_t\rvert$ or spiral update $\lvert x^* - x_t\rvert e^{bl}\cos(2\pi l) + x^*$.

Mixed continuous, integer, and categorical coordinates are mapped to the admissible learner domain by a projection/decoding map

$$\Gamma_\ell : \mathbb{R}^{q_\ell} \to \mathcal{H}_\ell.$$

Thus every candidate produced by Ψ_o is evaluated under the same search domain and fitness contract. Optimizer control parameters Θ_o (population size, iterations, crossover/mutation rates, swarm coefficients, trial limit, or leader/spiral controls) are distinct from learner hyperparameters $h \in \mathcal{H}_\ell$: Θ_o controls *how* the search proceeds, whereas h is the object being optimized.

D. Execution Contract, Invariants, and Sensitivity

One complete pass is represented compactly by

$$\tau_{d\ell or} = \langle Split, Transform, Select, Init, \\ (Generate, 10CV, Update)^T, Fit^*, Test \rangle.$$

For a metric m, the ten randomized passes are aggregated as

$$\overline{m}_{d\ell o} = \frac{1}{10}\sum_{r=1}^{10} m^{(r)}_{d\ell or},$$

so the inferential block is $b = (d, \ell)$, with six matched optimizer responses. The formal contract is reduced to four invariants sufficient for this study:

I1 (Isolation). Training and test indices are disjoint, and preprocessing, feature selection, optimization, and fitness use training information only.

I2 (Common contract). Within a fixed (d, ℓ, r), all optimizers use the same split, preprocessing, selector, learner, search domain, folds, and fitness criterion.

I3 (Feasibility). Every generated candidate is decoded/projected into $\mathcal{H}_\ell$.

I4 (Best-memory). Whenever an optimizer maintains an elite/best-so-far solution, $J^{MSE}_{cv}(h^*_{t+1}) \le J^{MSE}_{cv}(h^*_t)$.

These invariants imply test isolation and justify matched optimizer comparisons: within block $b = (d, \ell)$, optimizer identity is the treatment that changes while the experimental contract is fixed.

For RQ2, let $h_{b,o}$ be the selected mixed-type hyperparameter vector and δ the Gower distance. Configuration disagreement and performance response are

$$CD_b = \frac{2}{|\mathcal{O}|(|\mathcal{O}| - 1)}\sum_{i<j}\delta(h_{b,o_i}, h_{b,o_j}), \tag{3}$$

$$PSI_b = \frac{\max_o RMSE_{b,o} - \min_o RMSE_{b,o}}{\mathrm{median}_o(RMSE_{b,o})}. \tag{4}$$

Optimizer-specific configuration divergence is

$$CSI_{b,o} = \frac{1}{|\mathcal{O}| - 1}\sum_{o' \ne o}\delta(h_{b,o}, h_{b,o'}). \tag{5}$$

Spearman's $\rho(CD, PSI)$ tests whether greater disagreement in selected configurations is associated with greater predictive dispersion. This is explicitly a *configuration-output sensitivity* analysis; causal sensitivity to optimizer control parameters would require complete search trajectories or a controlled parameter-design experiment, which are outside the supplied data.

V. EMPIRICAL STUDY

The evaluation uses ten PROMISE datasets [25]: *ant-1*, *poi-2*, *camel-1*, *tomcat*, *ivy-2*, *velocity-1*, *jedit-4D*, *xalan-2*, *log4j-1*, and *xerces-1*. Eleven regressors are evaluated: Linear Regression, Decision Tree, Random Forest, Gradient Boosting, Lasso, Ridge, ElasticNet, KNN, SVR, MLP, and XGBRegressor. Their mathematical models and admissible hyperparameter spaces are specified in Section IV; feature selection uses `SelectFromModel` with `LassoCV`.

Six optimizers instantiate the inherited Optimization component: GA, PSO, ABC, CDEA, GWO, and WOA. The database contains all $10 \times 11 \times 6 = 660$ dataset–model–optimizer combinations. Each dataset–model pair is a matched block ($N = 110$), with the six optimizers as dependent treatments. The protocol preserves the published 70/30 split, log transformation of the response, standardization, LassoCV-based selection, and cross-validated hyperparameter evaluation. RMSE measures predictive error and recorded training time measures computational cost.

Data-integrity and leakage controls. The intended historical protocol uses a 70/30 holdout and training-only 10-fold CV, but the archive does not prove fold-local feature selection or duplicate isolation. An audit identified duplicate PROMISE records. We therefore ran a separate, reproducible sensitivity check on ten datasets: fixed Ridge, fold-local preprocessing, grouped 10-fold CV and disjoint grouped 70/30 holdout (20 comparisons; zero group overlaps). Exact-row deduplication increased test RMSE in nine datasets (one unchanged; mean $+0.01223$

on log1p defects). This diagnostic does not validate or correct the six-optimizer historical results.

Optimizer comparability. Numerical control parameters cannot be made literally identical across GA, PSO, ABC, CDEA, GWO, and WOA because their semantics differ (e.g., mutation/crossover rates versus inertia/acceleration coefficients or scout limits). Comparability is instead enforced at the experimental-contract level: the same outer split, training-only folds, preprocessing, feature selector, learner-specific domain $\mathcal{H}_\ell$, fitness definition, and final test protocol are held fixed within each matched block. A supplementary source-derived manifest records the six optimizers' function defaults and confirms calls without overrides (GA: 10 individuals, 5 generations; the other five: 6 agents, 4 iterations; ABC abandonment limit: 3). This documents the supplied implementation but does not establish that every archived experimental run used that source revision or the same random state. Causal control-parameter effects are not inferred.

Friedman tests ($\alpha = 0.05$) compare dependent treatments, with Kendall's $W = \chi_F^2/[N(k-1)]$ as effect size. Significant runtime differences are followed by Nemenyi comparisons. Dataset-level RQ3 tests use Holm multiplicity adjustment. RQ2 evaluates *configuration–output sensitivity*: Gower-based CSI quantifies differences among selected mixed-type hyperparameter dictionaries, PSI summarizes RMSE dispersion, and Spearman correlation tests their monotonic association. This design does not claim causal sensitivity of optimizer control parameters because complete search trajectories and controlled parameter perturbations are unavailable.

VI. Results and Analysis

The statistical analysis uses the 110 matched dataset–model blocks and all six optimizers available in the supplied results.

A. RQ1: Execution Time and Computational Performance

The overall Friedman test rejects equality of execution time among optimizers ($\chi_F^2 = 207.472$, $p = 7.15 \times 10^{-43}$), with Kendall's $W = 0.377$, indicating a substantial optimizer effect on computational time across the matched blocks. Average Friedman ranks (lower is faster) were GA=2.586, WOA=2.873, GWO=3.159, CDEA=3.309, PSO=3.314, and ABC=5.759. Thus, ABC is consistently the most computationally expensive treatment in matched comparisons. Nemenyi analysis indicates that ABC is significantly slower than every other optimizer ($p < 0.001$); GA also ranks faster than PSO ($p = 0.0455$) and CDEA ($p = 0.0479$).

Raw mean times were GA=6.241 s, PSO=2.979 s, CDEA=5.313 s, ABC=7.686 s, GWO=3.740 s, and WOA=2.322 s. The difference between raw means and rank order occurs because expensive learners such as forests and neural networks disproportionately affect arithmetic means, whereas Friedman ranks compare optimizers within the same dataset–model block.

Uncertainty reporting. From the 110 matched dataset–learner blocks, we computed percentile 95% bootstrap confidence intervals (10,000 block resamples; fixed seed) for each optimizer's mean runtime and RMSE; all twelve intervals and the reproducible calculation are supplied as supplementary material. These are estimated block-level intervals, not measurements from 6,600 new runs or pass-level intervals. No missing outcomes were imputed. Duplicate leakage and historical CV remain unverified.

TABLE III
Fastest Optimizer by Dataset (Mean Time Across 11 Models).

Dataset	Fastest	Mean time (s)
ant-1	WOA	2.314
camel-1	WOA	2.405
ivy-2	GA	1.775
jedit-4D	WOA	1.756
log4j-1	WOA	1.188
poi-2	WOA	1.239
tomcat	PSO	2.552
velocity-1	GA	2.707
xalan-2	WOA	3.824
xerces-1	WOA	2.405

Dataset-specific Friedman tests are significant for execution time in all ten datasets ($p \leq 0.0031$). This conclusion remains significant even under a conservative Bonferroni correction for ten tests ($0.0031 \times 10 = 0.031 < 0.05$), demonstrating that the computational differences are systematic rather than driven by a single project. WOA is the fastest by arithmetic mean in seven datasets, GA in two, and PSO in one.

B. RQ2: Configuration–Output Sensitivity

RQ2 is deliberately formulated as a configuration–output sensitivity question. The available experimental records contain the final hyperparameter dictionaries selected by each optimizer, but not controlled perturbations of optimizer control parameters or complete search trajectories. Consequently, the analysis asks whether different optimizers systematically select more divergent model configurations and whether such divergence is associated with a larger spread in predictive error. It does not claim causal sensitivity of GA/PSO/ABC/CDEA/GWO/WOA control parameters.

The mean CSI values (Eq. (5)) were GA=0.378, PSO=0.339, CDEA=0.331, ABC=0.326, GWO=0.335, and WOA=0.351. The Friedman test does not detect an optimizer effect on CSI ($\chi_F^2 = 8.769$, $p = 0.1187$), and Kendall's $W = 0.016$ indicates a very small effect. Thus, under the common search domains, the evidence does not support the claim that any optimizer systematically produces more divergent selected hyperparameter configurations than the others.

TABLE IV
BEST OBSERVED OPTIMIZER AND MODEL FOR EACH DATASET.

Dataset	Optimizer(s)	Model	Best RMSE	Friedman p
ant-1	GA	Random Forest	0.375138	0.1134
camel-1	GA	XGB	0.447986	0.0992
ivy-2	All six	Linear Regression	0.252629	0.1682
jedit-4D	PSO/CDEA/ABC/GWO/WOA	ElasticNet	0.385931	0.5797
log4j-1	CDEA	ElasticNet	0.384659	0.6665
poi-2	GA	MLP	**0.214542**	0.8294
tomcat	PSO	Random Forest	0.234000	0.3745
velocity-1	All six	Linear Regression	0.449690	0.8189
xalan-2	CDEA	Random Forest	0.259471	0.4030
xerces-1	CDEA	MLP	0.354127	0.4793

Across the 110 matched blocks, average pairwise configuration dispersion (Eq. (3)) was 0.343 and mean PSI (Eq. (4)) was 0.0588. In contrast to the weak optimizer effect on CSI, configuration dispersion and RMSE dispersion show a strong positive monotonic association (Spearman $\rho = 0.622$, $p = 4.12 \times 10^{-13}$). Therefore, blocks in which optimizers disagree more strongly about the selected hyperparameters also tend to exhibit larger predictive-performance dispersion. RQ2 is answered at two complementary levels: optimizer identity explains little of configuration divergence, whereas the magnitude of configuration disagreement is strongly related to the magnitude of performance disagreement. This is evidence of configuration–output sensitivity, not a causal global-sensitivity analysis of optimizer control parameters.

C. RQ3: Best Bio-Inspired Algorithm for Each Dataset

Table IV reports the lowest observed RMSE for each dataset over all 11 learners and six optimizers. These are descriptive winners; the last column reports the within-dataset Friedman p-value across the eleven matched learner blocks.

The best overall result is GA with MLP on *poi-2* (RMSE=0.214542). GA supplies the unique best result on three datasets, CDEA on three, and PSO on one; the remaining datasets contain ties. Nevertheless, no within-dataset Friedman test is significant at $\alpha = 0.05$ (minimum raw $p = 0.0992$). Applying Holm correction to the ten dataset-level tests makes the smallest adjusted value 0.992, so the conclusion is unchanged after multiplicity control. Consequently, Table IV identifies the best *observed* solution for each dataset but provides no inferential evidence that one optimizer statistically dominates the others within any individual dataset. The distinction between descriptive winner and statistically supported superiority is therefore explicit.

D. RQ4: Global Predictive Performance Differences

Across all 110 matched blocks, the Friedman test for RMSE gives $\chi^2_F = 10.687$ and $p = 0.05796$, with Kendall's $W = 0.019$. At $\alpha = 0.05$, the null hypothesis is not rejected; the small effect size is consistent with modest optimizer-related rank differences, although the p-value lies close to the decision threshold. Mean RMSE values were GA=0.38151, PSO=0.38009, CDEA=0.38356, ABC=0.37961, GWO=0.37866, and WOA=0.38073. The corresponding mean ranks were GA=3.286, ABC=3.236, GWO=3.514, CDEA=3.586, PSO=3.600, and WOA=3.777.

Thus, the experiment provides insufficient statistical evidence to conclude that predictive performance differs among the six optimizers at $\alpha = 0.05$; importantly, failure to reject the null hypothesis is not evidence of equivalence. In contrast, the runtime effect is clear and much larger ($W = 0.377$ versus $W = 0.019$ for RMSE). Combined with RQ2, the evidence supports selecting an optimizer using a joint criterion that considers runtime, implementation complexity, and configuration robustness in addition to predictive error.

E. Summary of Findings

Table V synthesizes the four RQ answers below.

TABLE V
ANSWERS TO THE RESEARCH QUESTIONS.

RQ	Main finding
RQ1	Execution time differs significantly; ABC is slowest by matched ranks, while WOA has the lowest raw mean time and is fastest on seven datasets.
RQ2	No optimizer systematically increases configuration divergence ($p = 0.1187$, $W = 0.016$), but configuration disagreement strongly tracks RMSE dispersion ($\rho = 0.622$, $p = 4.12 \times 10^{-13}$); this is configuration–output, not causal control-parameter sensitivity.
RQ3	Dataset-level descriptive winners vary among GA, CDEA, and PSO, with several ties; no dataset supports optimizer dominance, including after Holm multiplicity correction (minimum adjusted $p = 0.992$).
RQ4	There is insufficient evidence of global RMSE differences at $\alpha = 0.05$ ($p = 0.05796$, $W = 0.019$); this must not be interpreted as proof of optimizer equivalence.

VII. DISCUSSION

The four RQs reveal a consistent separation between computational and predictive behavior. For RQ1, optimizer identity has a substantial runtime effect ($W = 0.377$): ABC is slowest by matched ranks, whereas WOA has the lowest raw mean and is fastest on seven datasets. The difference between mean times and ranks is expected because costly learners disproportionately

influence arithmetic means, while Friedman ranks preserve within-block comparisons. Hence, computational efficiency is genuinely optimizer-dependent rather than an artifact of one project.

RQ2 provides a more precise sensitivity result than a conventional "parameter sensitivity" label would imply. Optimizer identity explains little configuration divergence ($p = 0.1187$, $W = 0.016$), yet configuration disagreement is strongly associated with RMSE dispersion ($\rho = 0.622$, $p = 4.12 \times 10^{-13}$). Thus, the evidence supports *configuration–output sensitivity*: when optimizers select more different learner hyperparameters, their predictive errors also tend to diverge. Causal sensitivity of mutation rates, inertia, colony limits, or other optimizer controls remains future work requiring controlled perturbations and repeated optimizer-level seeds.

RQ3 shows that descriptive winners are context-dependent: GA and CDEA each uniquely win three datasets and PSO one, with ties in the remainder. However, no dataset-level Friedman test is significant, including after Holm adjustment. Therefore, "best observed" must not be interpreted as statistical dominance. RQ4 reinforces this conclusion globally: RMSE differences do not reach $\alpha = 0.05$ ($p = 0.05796$, $W = 0.019$). This is insufficient evidence of a global optimizer effect, not proof of equivalence.

Together, the results strengthen the intended BIP 2025–BIP 2026 continuity. The predecessor supplies the architecture; this study formally specifies it, broadens its Optimization component, and stress-tests it empirically. Predictive quality is comparatively stable across optimizer substitutions, while runtime and local configuration behavior are more sensitive. In practice, optimizer choice should therefore consider computational cost and configuration robustness together with RMSE rather than selecting solely from a single best observed error.

Implications of RQ1. The runtime findings have a direct engineering interpretation. A search strategy that occasionally produces a low raw time is not necessarily the most consistently efficient strategy across heterogeneous learners. The matched-rank analysis is therefore the stronger basis for inference, while the arithmetic means retain practical value for estimating wall-clock cost. Their joint interpretation indicates that WOA is attractive when absolute mean time is important, GA is highly competitive in matched ordering, and ABC imposes a consistently larger computational burden. Because the predictive differences are comparatively small, these runtime differences can become decisive when tuning must be repeated across releases, datasets, or validation passes.

Implications of RQ2. The sensitivity analysis also clarifies where instability enters the framework. The very small CSI effect size means that no optimizer systematically occupies a more divergent region of the learner hyperparameter space. Nevertheless, the strong CSI–PSI association shows that the *degree* of disagreement within a dataset–learner block matters. This suggests a useful diagnostic interpretation: blocks with high configuration dispersion deserve additional validation because their predictive outcome is more dependent on which feasible configuration is selected. Conversely, low dispersion with similar RMSE indicates a locally robust region of the search space. This interpretation is consistent with the formal Generator–Evaluator relation, in which different feasible candidates are mapped to evaluated predictive responses under a common contract.

Implications of RQ3 and RQ4. The dataset-specific and global analyses answer different questions and should be read together. RQ3 asks whether a particular dataset exhibits an optimizer-specific advantage after controlling for learner identity; RQ4 asks whether such an effect persists over the complete set of matched blocks. Neither analysis supports statistical dominance. The presence of different descriptive winners is therefore evidence of contextual variation rather than a universal ranking. This matters for framework design: keeping Optimization modular is preferable to hard-coding one search algorithm as intrinsically superior. Where several optimizers reach the same best result, as occurs in some datasets, the rational choice can shift toward the less costly or simpler optimizer without sacrificing the best observed predictive value.

Formal-to-empirical correspondence. The empirical results can be read directly through the formal operators. The common transition Ψ_o changes only the search dynamics inside the Generator, while I2 fixes the Evaluator contract; this is precisely the controlled substitution tested by RQ1 and RQ4. The relation R_{GE} maps optimizer-generated configurations to J_{cv}^{MSE}, and R_{EG} returns that fitness to Ψ_o; RQ2 measures how differences in the resulting h^* propagate through R_{EP} and R_{pred} to held-out RMSE. Finally, I1 explains why those RMSE values can be interpreted as test outcomes rather than optimization feedback. Thus the significant runtime effect and weak RMSE effect are not detached empirical observations: they validate the architectural decision to make Ψ_o replaceable while preserving $\mathcal{E}$ and $\mathcal{P}$.

From a methodological perspective, the combination of significance tests and effect sizes is central to this interpretation. The extremely small runtime p-value is accompanied by a meaningful $W = 0.377$, whereas the near-threshold global RMSE result is accompanied by only $W = 0.019$. Reporting both avoids equating statistical detectability with practical importance. Likewise, Holm correction protects the dataset-level conclusions from overstatement when multiple RQ3 tests are considered simultaneously. These choices make the empirical extension more informative than a ranking based only on mean RMSE or isolated best configurations.

The results also have an architectural implication for the separation of concerns encoded by the framework.

Because the Generator and Evaluator expose a common configuration-and-fitness interface, the six search mechanisms can be compared without redefining the downstream prediction stage. This matters for extensibility: a new optimizer can be introduced as another realization of Ψ_o while preserving the data partition, learner definitions, evaluation criterion, and held-out prediction protocol. Conversely, if a future study changes preprocessing, the learner search space, or the evaluation metric at the same time as the optimizer, differences in the final error could no longer be attributed to optimizer substitution alone. The formal contract therefore acts not only as a description of the software architecture but also as an experimental-control mechanism.

This separation further clarifies how the runtime and RMSE findings should be used in practice. The absence of statistically supported RMSE dominance does not make optimizer selection irrelevant, because the cost of obtaining a competitive configuration differs markedly. For repeated tuning, a practitioner may reasonably prefer an optimizer with lower computational burden when its predictive performance is not distinguishable from more expensive alternatives under the same evaluation contract. Such a decision is conditional on the present search spaces and datasets; it is not a universal ranking of the six metaheuristics. In particular, a larger hyperparameter space, a more expensive learner, or a different stopping budget can alter the relative importance of exploration, exploitation, and evaluation cost. The appropriate interpretation is therefore a framework-level trade-off between predictive adequacy and optimization effort.

A second implication concerns reproducibility of comparative hyperparameter optimization. Meaningful replication requires more than reporting the winning configuration or the lowest observed RMSE. It requires preserving the partitioning rule, the exact objective optimized internally, the learner-specific domains, optimizer budgets, randomization policy, stopping conditions, and the final held-out evaluation procedure. The distinction introduced here between training-only J_{cv}^{MSE} and held-out RMSE is particularly important: feeding test performance back into Ψ_o would invalidate the invariant that supports the empirical comparison. Future implementations of the framework should therefore treat these elements as an explicit experimental manifest accompanying each run.

Finally, the findings suggest a useful boundary between algorithm comparison and algorithm explanation. The present design answers whether alternative optimizers, under a common contract, produce different computational and predictive outcomes; it does not identify which internal search dynamics caused a particular configuration to be selected. Explaining those dynamics would require trajectory-level evidence, repeated seeds, and controlled variation of optimizer parameters. Keeping these questions separate prevents descriptive differences in selected hyperparameters from being overinterpreted as causal effects of particular control parameters and provides a clearer progression for subsequent studies built on the framework.

The empirical pattern is also consistent with the role assigned to the Evaluator in the formal specification. All optimizers ultimately compete through the same learner-specific objective, so different search trajectories can terminate at distinct configurations that nevertheless occupy regions of similar predictive quality. This offers a plausible framework-level explanation for observing configuration disagreement without corresponding global RMSE dominance. The study does not claim that these regions form equivalent optima, because establishing that property would require denser characterization of the response surface. It does show, however, that configuration identity alone is insufficient for judging the practical quality of a search outcome; the configuration must be interpreted together with its predictive error and the computational effort required to obtain it.

For software defect-density prediction, this point is consequential because the framework is intended to support heterogeneous learners rather than one fixed model family. Search difficulty can vary substantially across learner domains, and the optimizer that is economical for one learner need not preserve the same advantage for another. The matched dataset–learner blocks used here are therefore an important unit of analysis: they preserve local comparability while allowing the global tests to summarize behavior across heterogeneous modeling conditions. This design choice complements the architectural modularity and provides a disciplined basis for extending the comparison with additional learners or optimizers without changing the interpretation of the existing evidence.

VIII. Threats to Validity

Construct validity. RMSE is appropriate for regression-based defect-density prediction and penalizes large deviations, but it does not capture every notion of predictive utility. MAE or release-oriented cost measures could provide complementary evidence. Likewise, execution time is an empirical computational indicator rather than a hardware-independent complexity measure; absolute values may vary with platform, libraries, parallelization, and system load. The matched experimental contract nevertheless supports fair within-study optimizer comparisons.

Internal validity. Common preprocessing, feature selection, learner search domains, train/test partitioning, and cross-validation reduce confounding. Each dataset–learner pair is treated as a matched block, so optimizer identity is the treatment that changes within a comparison. Still, implementation-specific details and random initialization can influence convergence and runtime. The supplementary CSV audit finds duplicate and potential near-duplicate records in the supplied PROMISE inputs;

because historical split assignments are unavailable, the effect on published test errors cannot be quantified without rerunning the study. Moreover, the available records contain final selected hyperparameter dictionaries and aggregated outcomes rather than complete search trajectories or controlled perturbations of optimizer controls. RQ2 therefore measures *configuration–output sensitivity*; it does not establish causal sensitivity of mutation probability, inertia, colony limits, differential weights, or related internal parameters.

Conclusion validity. Friedman tests respect the dependent block structure, Kendall's W quantifies effect magnitude, and post-hoc/multiplicity procedures are applied where required. This reduces the risk of claiming superiority from isolated means or minimum RMSE values. The global RMSE result ($p = 0.05796$) is deliberately interpreted as insufficient evidence of differences, not as proof of equivalence. An equivalence design with prespecified practical margins would be required for the latter claim. Dataset-specific RQ3 tests also contain only eleven learner blocks, limiting power for small effects. Block-bootstrap 95% confidence intervals are supplied for mean runtime and RMSE across the 110 matched blocks; pass-level uncertainty remains unidentifiable from the aggregate export. Future replication should retain all randomized-pass observations.

External validity. The ten PROMISE datasets provide heterogeneous benchmark cases but cannot represent every language, repository scale, defect-reporting process, or industrial environment. This single benchmark family is an explicit limitation: the conclusions therefore apply to the evaluated PROMISE setting and should be replicated on independent industrial systems, additional benchmark families and releases, cross-project scenarios, and broader learner/search spaces. Such replications are particularly relevant because optimizer behavior may change as the dimensionality and geometry of the hyperparameter space change.

IX. Conclusions and Future Work

BIP 2026 is a direct continuation, not a reproduction, of the IEEE BIP 2025 framework. The predecessor established the Generator–Evaluator–Predictor architecture and its White-Box realization; this study preserves that contract while advancing the research line through (i) a hierarchical mathematical specification of its components, elements, relations, operators, and invariants, (ii) generalization of Optimization from the GA-centered realization to six bio-inspired optimizers, and (iii) an expanded 660-record evaluation organized into 110 matched dataset–learner blocks. BIP 2025 therefore provides the architectural baseline, whereas BIP 2026 contributes formal specification, algorithmic generalization, and extended inferential validation.

The four RQs yield a coherent result. RQ1 identifies a substantial optimizer effect on runtime ($p = 7.15 \times 10^{-43}$, $W = 0.377$), with ABC costly in matched rankings and WOA obtaining the lowest raw mean time. RQ2 shows that optimizer identity explains little configuration divergence ($W = 0.016$), yet configuration disagreement is strongly associated with RMSE dispersion ($\rho = 0.622$, $p = 4.12 \times 10^{-13}$), supporting configuration–output sensitivity. RQ3 shows that descriptive winners depend on the dataset and that none becomes inferentially dominant after multiplicity control. RQ4 finds insufficient evidence of global RMSE differences at $\alpha = 0.05$ ($p = 0.05796$, $W = 0.019$), which must not be interpreted as optimizer equivalence.

Collectively, these results support the modular design inherited from BIP 2025. No optimizer is justified as universally superior in predictive quality, while optimizer substitution can materially affect computational cost. Optimization should therefore remain replaceable and be selected using runtime, configuration robustness, implementation complexity, and predictive error jointly rather than minimum RMSE alone. This is especially relevant when tuning is repeated across releases or projects, where small predictive differences may coexist with much larger computational differences.

The formal and empirical contributions also reinforce one another. The set-based specification makes the optimizer a replaceable element governed by common Generator–Evaluator–Predictor relations, while the experiments show why that abstraction is useful: changing the optimizer preserves broadly similar predictive behavior but can alter computational behavior substantially. This correspondence between specification and evidence strengthens the interpretation of BIP 2026 as a formal and experimental extension of the original framework rather than an independent benchmark.

An additional practical implication concerns how the framework should be used in subsequent studies. The results discourage selecting an optimizer from a single best RMSE value without considering the stability and cost of reaching that value. Instead, the framework supports a staged decision process in which predictive adequacy is established first and computational criteria are then used to discriminate among alternatives whose predictive differences are small or unsupported statistically. Under this interpretation, the optimization component becomes a configurable engineering choice rather than a fixed property of the prediction model. This preserves the original architectural intent while making the extended framework more suitable for repeated experimentation, replication, and controlled substitution of future search mechanisms.

The present study also clarifies what evidence would be needed to move beyond the current conclusions. Demonstrating optimizer equivalence would require an equivalence or non-inferiority design with practically meaningful margins defined before analysis; demonstrating causal parameter sensitivity would require controlled

perturbation of optimizer controls rather than comparison of the final configurations they select. Likewise, claims about convergence reliability would require pass-level trajectories and repeated seeds. These distinctions delimit the contribution without weakening it: BIP 2026 establishes comparative behavior under a common contract and identifies the additional experimental evidence required for stronger causal or equivalence claims.

Future work should retain row-level split provenance, verified optimizer-control manifests, complete trajectories, pass-level outcomes, and repeated optimizer-level seeds to analyze convergence, run-to-run variance, and stopping behavior. Controlled variation of optimizer parameters would enable causal global-sensitivity methods such as Sobol indices or variance decomposition, complementing the configuration–output sensitivity established here. The formal model can also be extended with optimizer-specific invariants suitable for implementation verification. Finally, multi-objective optimization should jointly consider error, execution time, and configuration stability, while replication on industrial and cross-project datasets should test the external scope of the present conclusions.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

AI-assisted tools were used for editorial support, mathematical notation, and organization of the formal specification. All statistical values reported in the paper were computed from the supplied experimental results and were reviewed by the authors; AI tools were not used to fabricate experimental observations.

References

[1] T. Menzies, J. Greenwald, and A. Frank, "Data mining static code attributes to learn defect predictors," *Software Engineering, IEEE Transactions on*, vol. 33, no. 1, pp. 2–13, Jan 2007.

[2] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the nasa software defect datasets," *Software Engineering, IEEE Transactions on*, vol. 39, no. 9, pp. 1208–1215, 2013.

[3] J. Murillo-Morera, R. Fuentes-Fernández, A. Calvo Vargas, A. Cambronero Delgado, J. Jadán-Guerrero, and S. A. Arias, "A framework for hyperparameter configuration in the context of software defect density prediction using bio-inspired optimization algorithms," in *2025 IEEE 7th International Conference on BioInspired Processing (BIP)*, 2025.

[4] V. R. Basili, "Software modeling and measurement: the goal/question/metric paradigm," 1992.

[5] V. Podgorelec, "Improved mining of software complexity data on evolutionary filtered training sets," *WSEAS Trans. Info. Sci. and App*, vol. 6, pp. 1751–1760, 2009.

[6] R. Malhotra, "Comparative analysis of statistical and machine learning methods for predicting faulty modules," *Applied Soft Computing*, vol. 21, pp. 286–297, 2014.

[7] P. Singh and S. Verma, "Empirical investigation of fault prediction capability of object oriented metrics of open source software," in *Computer Science and Software Engineering (JCSSE), 2012 International Joint Conference on*. IEEE, 2012, pp. 323–327.

[8] H. Wang, T. M. Khoshgoftaar, and A. Napolitano, "Software measurement data reduction using ensemble techniques," *Neurocomputing*, vol. 92, pp. 124–132, 2012.

[9] J. Murillo, C. Quesada, and M. Jenkins, "Software fault prediction: A systematic mapping study," in *XVIII Ibero-American Conference on Software Engineering*. CibSE, 2015, pp. 50–64.

[10] Q. Song, Z. Jia, M. Shepperd, S. Ying, and J. Liu, "A general software defect-proneness prediction framework," *Software Engineering, IEEE Transactions on*, vol. 37, no. 3, pp. 356–370, 2011.

[11] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," *IEEE Transactions on Software Engineering*, vol. 34, no. 4, pp. 485–496, 2008.

[12] S. Huda, S. Alyahya, M. M. Ali, S. Ahmad, J. Abawajy, H. Al-Dossari, and J. Yearwood, "A framework for software defect prediction and metric selection," 1 2018.

[13] K. Bashir, T. Li, and W. Chubato, "An empirical study for enhanced software defect prediction using a learning-based framework," *International Journal of Computational Intelligence Systems*, vol. 12, p. 282, 11 2018.

[14] M. Hesamolhokama, A. Shafiee, M. Ahmaditeshnizi, M. Fazli, and J. Habibi, "Sdperl: A framework for software defect prediction using ensemble feature extraction and reinforcement learning," 2024.

[15] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, J. Thomas, L. Ullmann, J. Becker, M. Boulesteix, D. Deng, and M. Lindauer, "Hyperparameter optimization: Foundations, algorithms, best practices and open challenges," *arXiv preprint arXiv:2107.05847*, nov 2021.

[16] J. Murillo-Morera and M. Jenkins, "A software defect-proneness prediction framework: A new approach using genetic algorithms to generate learning schemes," in *The 27th International Conference on Software Engineering and Knowledge Engineering, SEKE 2015, Wyndham Pittsburgh University Center, Pittsburgh, PA, USA, July 6-8, 2015*, 2015, pp. 445–450.

[17] Z. Jakšić, S. Devi, O. Jakšić, and K. Guha, "A comprehensive review of bio-inspired optimization algorithms including applications in microelectronics and nanophotonics," *Biomimetics*, vol. 8, no. 3, p. 278, Jun. 2023.

[18] J. Murillo-Morera, C. Quesada-López, C. Castro-Herrera, and M. Jenkins, "An empirical validation of an automated genetic software effort prediction framework using the isbsg dataset," 2016.

[19] J. Murillo-Morera, C. Castro-Herrera, J. Arroyo, and R. Fuentes-Fernández, "An Empirical Validation of Learning Schemes using an Automated Genetic Defect Prediction Framework," July 2016, in press Springer.

[20] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of the IEEE International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948.

[21] D. Karaboga, "An idea based on honey bee swarm for numerical optimization," Erciyes University, Engineering Faculty, Computer Engineering Department, Tech. Rep. TR06, 2005.

[22] Y. Wang, Z. Cai, and Q. Zhang, "Differential evolution with composite trial vector generation strategies and control parameters," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 55–66, 2011.

[23] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014.

[24] S. Mirjalili and A. Lewis, "The whale optimization algorithm," *Advances in Engineering Software*, vol. 95, pp. 51–67, 2016.

[25] T. Lamba, Kavita, and A. Mishra, "Optimal metrics selection for software defect prediction," *International Journal of Data Mining and Emerging Technologies*, vol. 7, p. 82, 1 2017.